

# Project 03 예약 SaaS — Phase 1-4 학습정리

## EF Core 연결 + DbContext + xmin 낙관적 잠금 + 마이그레이션 (엔티티가 실제 테이블이 되다)

이 문서는 "내가 한 것" + "남들이 자주 겪는 문제 + 우리 케이스"를 함께 수록한다. 1-4는 함정이 유독 많았던 단계라 함정 섹션이 특히 두껍다.

## 0. Phase 1-4는 무엇인가

1-3에서 만든 순수 C# 엔티티(Tenant, User)를 실제 PostgreSQL 테이블로 연결한다. Domain(엔티티)과 DB가 처음으로 이어지는 단계이며, EF Core라는 ORM이 그 다리 역할을 한다.

### 1-4에서 한 일 (7단계)

- |                           |                                |
|---------------------------|--------------------------------|
| ① EF Core + Npgsql 패키지 설치 | (Infrastructure, 전부 10 패밀리)    |
| ② DbContext 작성            | (엔티티 ↔ DB 통역사, Infrastructure) |
| ③ xmin 낙관적 잠금 설정          | ([Timestamp] uint Version)     |
| ④ 연결 문자열 설정               | (User Secrets, 비번을 코드 밖에)      |
| ⑤ 첫 마이그레이션 생성             | (엔티티 ↔ SQL 설계도)                |
| ⑥ 마이그레이션 DB 적용            | (실제 테이블 생성)                    |
| ⑦ 자동 적용 패턴                | (앱 시작 시 db.Database.Migrate()) |

## 1. 핵심 개념

### ORM과 DbContext

- **ORM (Object-Relational Mapping)**: 객체(C# 클래스)와 관계형 DB(테이블)를 자동으로 오가게 하는 도구. user 객체 저장 → INSERT SQL 자동 변환.
- **EF Core vs Dapper**: 03은 EF Core(관계형 도메인에 강함, 자동화 많음). IPlus·05는 Dapper(생 SQL, 성능·읽기전용에 강함). 두 ORM을 다 쓰는 것 자체가 포트폴리오 강점.
- **DbContext**: EF Core의 중앙 관제소. ①테이블 선언( DbSet ) ②매핑 규칙( OnModelCreating ) ③DB 작업 통로. Infrastructure 계층에 위치 (DB 기술이므로).

### DI(의존성 주입)와 Composition Root

- DbContext는 연결 정보를 직접 만들지 않고 생성자로 주입받는다( : base(options) ).
- 실제 주입은 API의 Program.cs에서: AddDbContext<...>(o => o.UseNpgsql(연결문자열)).
- 이 "의존성을 조립하는 진입점"을 Composition Root라 하며, 클린 아키텍처에서 Presentation(API)의 정당한 역할이다. → API가 Infrastructure를 참조하는 것은 위반이 아니다(후술).

## 2. 단계별 상세

### ① 패키지 설치 (Infrastructure)

```
dotnet add Reservation.Infrastructure package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add Reservation.Infrastructure package Microsoft.EntityFrameworkCore.Design
```

- **Npgsql.EntityFrameworkCore.PostgreSQL** (10.0.2): PostgreSQL용 EF Core 프로바이더. EF Core 본체도 함께 끌어옴.
- **Microsoft.EntityFrameworkCore.Design** (10.0.9): 마이그레이션 도구용. 런타임엔 안 쓰이고 개발 시 마이그레이션 생성 때만 사용.
- EF Core 10 + PG 18 조합 이점: Guid를 .NET에서 UUIDv7로 생성 → 인덱스에 유리.

## ② DbContext 작성

```
using Microsoft.EntityFrameworkCore;
using Reservation.Domain.Entities;

namespace Reservation.Infrastructure.Persistence;

public class ReservationDbContext : DbContext
{
    public ReservationDbContext(DbContextOptions<ReservationDbContext> options)
        : base(options) { }

    public DbSet<Tenant> Tenants => Set<Tenant>();
    public DbSet<User> Users => Set<User>();
}
```

- `: DbContext` 상속으로 DB 기능 물려받음.
- 생성자가 `DbContextOptions` 를 받아서 base로 전달 → 연결 정보를 밖에서 주입받는 구조(DI).
- `DbSet<T>` = 그 엔티티에 대응하는 테이블 선언. 이름(복수형)이 테이블명이 됨.

## ③ xmin 낙관적 잠금

**낙관적 잠금(Optimistic Lock):** 잠그지 않고, 저장 시점에 "버전이 그대로인가?"만 확인해 충돌 감지. 웹 멀티유저에 적합(비관적 잠금은 대기가 생겨 불리). - PostgreSQL은 모든 행에 숨은 시스템 컬럼 `xmin` 을 가짐 → 행이 수정될 때마다 자동 변경 → **버전 번호로 이상적**. - **MSSQL rowversion**  
↔ **PostgreSQL xmin** = 같은 개념, 다른 구현. (11년 MSSQL 경력 + PG 신규가 만나는 지점)

구현 (BaseEntity에 한 번만):

```
using System.ComponentModel.DataAnnotations;

public abstract class BaseEntity
{
    public Guid Id { get; set; }
    public DateTime CreatedAt { get; set; }

    [Timestamp]
    public uint Version { get; set; } // xmin에 자동 매핑
}
```

- 주의: 구버전 방식 `엔티티.UseXminAsConcurrencyToken()` 은 Npgsql 7.0+에서 제거됨. **표준 EF 방식** `[Timestamp] uint` 로 대체(공식 문서 확인). BaseEntity에 두면 모든 엔티티가 자동 상속.

충돌 감지 범위: 03은 감지까지( `DbUpdateConcurrencyException` ). 해결 고도화(필드 단위 3-way merge)는 04로.

## ④ 연결 문자열 (User Secrets)

비밀번호를 코드/저장소에 두지 않기 위해 환경별로 분리:

| 환경        | 저장 위치                       | 이유         |
|-----------|-----------------------------|------------|
| 로컬 개발     | User Secrets (프로젝트 폴더 밖)    | git에 안 올라감 |
| 배포(Azure) | Container App secret / 환경변수 | 코드에 없음     |

```
dotnet user-secrets init --project Reservation_API
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Host=...;Database=reservationdb;Username=reservation_app;Password=***;SslMode=Require"
```

- `SslMode=Require` : Azure PostgreSQL은 암호화 연결 필수.
- Program.cs에서 읽어 주입:

```
var cs = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ReservationDbContext>(o => o.UseNpgsql(cs));
```

- `Configuration` 은 appsettings/User Secrets/환경변수를 **자동으로 다 뒤져** 값을 찾음 → 코드는 그대로, 출처만 환경 따라 달라짐.

## ⑤⑥ 마이그레이션 생성·적용

```
# 생성 (엔티티 → SQL 설계도)
dotnet ef migrations add InitialCreate --project Reservation.Infrastructure --startup-project Reservation_API

# 적용 (설계도 → 실제 테이블)
dotnet ef database update --project Reservation.Infrastructure --startup-project Reservation_API
```

- **--project** (마이그레이션 위치=Infrastructure) + **--startup-project** (설정 위치=API) 둘 다 필요 → 클린 아키텍처로 계층을 나눴기 때문.
- 마이그레이션 = DB 스키마 변경 이력을 코드로 관리(Git이 코드 이력 관리하듯).
- 생성 후 반드시 파일을 눈으로 확인(아래 함정 참고).

## ⑦ 자동 적용 패턴

```
var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<ReservationDbContext>();
    db.Database.Migrate(); // 안 적용된 마이그레이션 자동 적용
}
```

- 컨테이너는 뜰 때마다 빈 DB일 수 있음 → 앱 시작 시 스스로 스키마 보장.
- 이미 최신이면 **No migrations were applied** 로그와 함께 조용히 넘어감(여러 번 커도 안전).
- 02 Todo에서 검증한 패턴, 03·04 동일 적용.

## 3. ★ 흔한 함정 (남들이 자주 겪는 것 + 우리 케이스)

| #  | 함정                            | 증상                                                  | 우리 케이스 / 해결                                                                                        |
|----|-------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 1  | EF 패치 버전 충돌                   | Microsoft.EntityFrameworkCore 10.0.4 vs 10.0.9 충돌   | 메이저(10)는 맞아도 패치가 어긋남. 최상위에 <b>--version 10.0.9</b> 명시하면 그게 이겨 통일                                   |
| 2  | transitive 패키지 취약점            | NU1903 Microsoft.OpenApi 2.0.0 취약점 (CVE-2026-49451) | <b>dotnet list --include-transitive</b> 로 출처 추적 → <b>패치 버전(2.7.5)</b> 최상위 명시로 덮어씀                  |
| 3  | 구버전 API 사용                    | UseXminAsConcurrencyToken 정의 없음                     | Npgsql 7.0+에서 방식 변경. 공식 문서 확인 → <b>[Timestamp] uint</b> 로 대체                                       |
| 4  | EF Design이 startup 프로젝트에 없음   | startup project doesn't reference ...Design         | 클린 아키텍처 계층 분리 특유. <b>API에도 Design 추가</b> (위반 아님, 후술)                                               |
| 5  | 생성자에서 파라미터에 재대입               | 값이 증발(컴파일 에러 없음!)                                   | <b>name = name.Trim() ( ) → Name = name.Trim() ( )</b> . User·Tenant 둘 다 있었음. 속성명과 파라미터명이 비슷할 때 빈출 |
| 6  | 백킹 필드가 컬럼명으로 노출               | DB 컬럼이 <b>m_Email</b> 로 생성                          | public 속성명은 표준으로( <b>Email</b> ). <b>m_</b> 은 private 필드 접두사지 속성명 아님                               |
| 7  | 연결 문자열 형식 오류                  | Format ... does not conform ... index 123           | 비번의 특수문자( ; 등)가 구분자와 충돌. ; ' = 없는 비번으로                                                             |
| 8  | DB 비번 ≠ 연결 문자열 비번             | 28P01 password authentication failed                | DB 비번 바꾸면 User Secrets도 같이. <b>양쪽을 같은 값으로 재설정</b>                                                  |
| 9  | PasswordHash에 .Trim()         | 해시 손상 가능 → 로그인 비교 실패                                | 해시는 받은 그대로 저장. <b>.Trim()</b> 은 Email 같은 사용자 입력에만                                                  |
| 10 | __EFMigrationsHistory 조회 fail | 첫 실행 시 빨간 <b>fail</b> 로그                            | <b>정상</b> (이력 테이블이 아직 없어서). 뒤에 <b>Done. / up to date</b> 뜨면 성공. 최종 결과를 볼 것                         |
| 11 | DateTime.Now 사용               | 서버 지역시간 저장(westus2=미국)                              | <b>DateTime.UtcNow</b> . (User 생성자에 또 등장 — 반복 실수)                                                  |

에러 코드로 원인 구분 (합정 7 vs 8): `Format ... index N` 은 연결 문자열 형식(파싱 전) 문제, `28P01` 은 인증 실패(DB 도달 후) 문제. 전자는 문자열 구조, 후자는 비번 값. 에러 성격이 완전히 다르다.

## 4. EF Design을 API에 추가하는 게 클린 아키텍처 위반인가? (핵심 Q&A)

아니다. 두 가지 이유: 1. **Design**은 런타임 코드가 아니라 개발 도구. 마이그레이션 생성 때만 쓰이고 실행 앱의 의존성이 아니다(CAD가 완공된 건물에 없듯). 2. **API(Presentation)**는 **Composition Root**로서 이미 Infrastructure를 참조해 DB를 조립한다. 이는 클린 아키텍처가 진입점에 부여하는 정당한 책임.

클린 아키텍처가 막는 건 **Domain·Application**이 **DB 구현에 의존하는 것**이지, **진입점이 조립을 위해 참조하는 것**이 아니다.

면접 답변: "Design은 개발 도구라 런타임 의존성이 아니고, API는 Composition Root로서 의존성을 조립하는 역할이라 Infrastructure 참조가 정당하다. 클린 아키텍처가 금지하는 건 안쪽 계층(Domain/Application)의 DB 의존이지 진입점의 조립이 아니다."

## 5. 마이그레이션 파일 읽는 법 (반드시 확인)

생성된 `..._InitialCreate.cs` 의 `Up()` 을 열어 의도대로 번역됐는지 확인: - `Id : uuid` (Guid) - `Email : character varying(256)` ([MaxLength(256)] 반영) — `m_` 없어야 - `PasswordHash : text` (해시는 길이 다양 → 무제한 OK) - `CreatedAt : timestamp with time zone` (UtcNow와 정합) - `xmin : xid, rowVersion: true` (낙관적 잠금) - `TenantId : uuid + FK( FK_Users_Tenants_TenantId )` + 인덱스 (`IX_Users_TenantId`)

교훈: "마이그레이션은 만들고 끝이 아니라 반드시 읽어본다." 이 습관 덕에 `m_Email` 노출과 `varchar` 미적용을 사전에 잡았다.

## 6. 이번 단계의 결과물 (DB 실제 구조)

`reservationdb` / `public` 스키마: - **Tenants** (Id, Name varchar(200), CreatedAt, xmin) - **Users** (Id, Email varchar(256), PasswordHash text, CreatedAt, xmin, TenantId) - **\_\_EFMigrationsHistory** (EF 관리용 — 어떤 마이그레이션을 적용했는지 기록)

FK: Users.TenantId → Tenants.Id (onDelete Cascade — 추후 정책 재검토 메모)

## 7. 면접에서 말할 수 있는 포인트

- 낙관적 잠금 플랫폼 독립 설계 — MSSQL rowversion과 PG xmin이 같은 개념임을 알고, PG에선 `[Timestamp] uint` 로 구현. 감지는 03, 해결 고도화는 04.
- 의존성 취약점 대응 — NU1903 경고 → transitive 추적 → 패치 버전 명시. 공급망 보안 감각.
- 버전 정합성 관리 — EF 패치 충돌을 최상위 명사로 통일. 도구/런타임 버전 일치.
- 비밀 관리 — User Secrets(로컬) · 환경변수(배포)로 연결 문자열 분리. 코드·저장소에 비번 없음.
- Composition Root 이해 — API의 Infrastructure 참조가 클린 아키텍처상 정당함을 근거로 설명.
- 자동 마이그레이션 — 컨테이너 시작 시 스키마 자동 보장(명등).
- 디버깅 감각 — 연결 문자열 형식 오류 vs 28P01 인증 실패를 에러로 구분.

## 8. 다음 — Phase 1-5 (JWT 인증)

이제 사용자가 로그인하고 신원을 증명하는 인증을 만든다. - **Access Token + Refresh Token 분리** (순수 stateless 아님). - **Refresh Token은 httpOnly 쿠키 + DB 저장(stateful)** → 무효화·로그아웃 가능(구글/네이버 스타일). - **Token Rotation + 재사용 탐지** 구현(개념 학습 포함) — 가산 점 카드. - 비밀번호 해싱(여기서 PasswordHash에 실제 해시가 채워짐). - CSRF 방어(SameSite). - 클린 아키텍처로: 인증 로직은 Application, 토큰 저장은 Infrastructure, 엔드포인트는 API.

---

Phase 1-4 학습정리 끝. 작성 기준: EF Core 10.0.9 / Npgsql 10.0.2 / PostgreSQL 18 / .NET 10.